

# Documentazione Progetto Reti Informatiche

Matteo Canessa 659435

## File e Compilazione

Il progetto è costituito da 6 file sorgente:

- `utente.c` - Implementazione del client
- `lavagna.c` - Implementazione del server
- `funzioni_x_msg.c` - Funzioni condivise per la comunicazione
- `strutture_utente.h` - Definizioni strutture dati del client
- `strutture_lavagna.h` - Definizioni strutture dati del server
- `funzioni_x_msg.h` - Header delle funzioni di comunicazione

I file `funzioni_x_msg` sono utilizzati sia dal client sia dal server e contengono le funzioni `send_message()` e `recv_message()`, che implementano il meccanismo di invio e ricezione dei messaggi (viene ripreso dopo). Nella cartella del progetto è presente uno script `compila.sh` che permette di compilare tutti i file sorgente con le flag richieste. È presente anche un file `card.txt` che contiene le card iniziali della lavagna, che verranno caricate all'avvio del server.

## Client-Server

Architettura `client-server multi-thread`. Il protocollo utilizzato è il TCP con `socket bloccanti`; l'utilizzo del TCP semplifica la gestione della comunicazione, in quanto garantisce affidabilità e ordine dei messaggi, inoltre qualora ci siano disconnessioni queste vengono rilevate. Il protocollo UDP non è stato scelto in quanto la velocità garantita non rappresenta un trade-off sufficiente rispetto alle qualità offerte dal TCP. I socket bloccanti sono stati scelti per semplificare la logica di attesa dei messaggi, il server può gestire contemporaneamente più client tramite thread dedicati, mentre il client può attendere in lettura messaggi asincroni dalla lavagna senza compromettere l'esecuzione. È stato definito un formato standard per lo scambio delle informazioni testuali: ogni messaggio viene preceduto da un campo di lunghezza di 2 byte in formato binario (tipo `unsigned short`), convertito in network order tramite `htons/ntohs` per evitare problemi di endianess. Questo approccio permette di gestire messaggi di lunghezza variabile in modo efficiente, evitando l'invio di bytes in eccesso o limitazioni fisse sulla dimensione dei messaggi. I campi all'interno del messaggio sono separati dal carattere `|`, definito nella costante `CARATTERE_SEPARATORE`. Il protocollo utilizzato permette di evitare la complessità di un protocollo interamente binario che richiederebbe serializzazione delle strutture e gestione della compatibilità tra architetture diverse; allo stesso tempo semplifica il parsing rispetto a un protocollo testuale puro con delimitatori. Gli eventuali segnali SIGPIPE che la send può emettere sono evitati tramite la flag `MSG_NOSIGNAL`, prevenendo terminazioni inaspettate del processo in caso di scrittura su socket chiuso. La scelta del `multi-thread` è stata preferita rispetto all'I/O multiplexing per semplificare la gestione di operazioni bloccanti su più socket e rispetto alla `fork()`, che comporta la duplicazione dell'intero spazio di memoria che va in contrasto con la scelta di gestione della memoria adottata. I thread vengono `detached` per permettere la deallocazione automatica delle risorse al termine, evitando di trasformarli in thread zombie.

## Server (Lavagna)

### Struttura dati

Le strutture utilizzate per l'implementazione della Lavagna sono principalmente quattro struct:

`st_lavagna`, `st_colonna`, `st_card` e `info_utente`. La gestione in memoria di queste strutture è dinamica, con allocazione e deallocazione in base alle esigenze; questa scelta è stata effettuata per ottimizzare l'uso della memoria e per permettere una maggiore flessibilità nella gestione delle card e degli utenti, in quanto il numero di card e utenti connessi può variare nel tempo. La struttura `info_utente` contiene le informazioni relative ad ogni utente connesso, come la porta, il socket descriptor (utilizzato per la comunicazione), la flag di occupazione (indica se l'utente ha una card assegnata) e la flag `pong_ricevuto` (utilizzata per il meccanismo di ping/pong). L'identificazione degli utenti tramite porta invece che username semplifica l'architettura P2P, poiché la porta serve sia come identificativo univoco che come indirizzo per le connessioni dirette. L'uso di `realloc()` per card e utenti permette scalabilità ma introduce overhead ad ogni inserimento/rimozione, accettabile dato il numero limitato di utenti previsti e dalla bassa frequenza di tali operazioni.

### Approccio

L'approccio adottato per la gestione della lavagna è stato quello di utilizzare un paradigma multi-thread, in quanto il server deve: gestire contemporaneamente più utenti che accedono e interagiscono con la stessa lavagna condivisa, gestire i comandi inseriti da terminale e inviare periodicamente messaggi di ping agli utenti con card in DOING per verificare la loro disponibilità. Per ogni connessione in ingresso viene creato un nuovo thread che gestisce i comandi

in arrivo dal client. L'accesso alla lavagna è protetto da un mutex globale (`accesso_lavagna`) che garantisce la mutua esclusione nelle operazioni sulle strutture condivise. Una variabile condition (`lavagna_aggiornata`) viene utilizzata per implementare un meccanismo di notifica: ogni volta che lo stato della lavagna cambia (spostamento o completamento di una card), il thread che ha effettuato la modifica segnala la condition, risvegliando il thread di broadcast che invia lo stato aggiornato a tutti i client connessi.

## Client (Utente)

### Struttura dati

La struttura dati principale lato client è la struct `CARD_ASSEGNATA`, che contiene le informazioni relative alla card attualmente assegnata all'utente: ID della card, testo dell'attività, array dinamico delle porte degli altri utenti (per le review P2P), numero di utenti connessi, contatore delle review ricevute, e flag per tracciare l'invio di `ACK_CARD` e `CARD_DONE`. La scelta di mantenere localmente questi dati permette di validare i comandi dell'utente senza interrogare continuamente il server, riducendo il traffico di rete e migliorando la responsività dell'interfaccia. Sono inoltre presenti variabili globali per tracciare lo stato del client: `hello_eseguito` (indica se l'utente si è registrato), `ping_ricevuto` (flag per il meccanismo di risposta al ping).

### Approccio

Il client implementa un'architettura multi-thread composta da tre componenti principali: il thread principale gestisce l'interfaccia utente e l'invio dei comandi; un thread di ascolto (`gestione_ascolto`) riceve messaggi asincroni dalla lavagna (card assegnate, ping, stato aggiornato della lavagna); un thread server P2P (`server_p2p`) rimane in ascolto sulla porta dell'utente per ricevere richieste di review da altri utenti.

### Comunicazione P2P

Quando un utente esegue il comando `REVIEW_CARD`, richiede prima la lista degli utenti connessi al server per avere dati aggiornati. Per ogni utente nella lista, viene creato un thread dedicato che stabilisce una connessione TCP diretta sulla porta dell'utente destinatario. L'incremento del contatore `review_ricevute` è protetto da un mutex poiché più thread possono modificarlo contemporaneamente. La scelta della comunicazione P2P diretta per le review è stata preferita rispetto a un approccio mediato dal server per ridurre il carico sulla lavagna e distribuire il traffico di rete. Tuttavia, questo introduce la necessità di gestire porte univoche per ogni utente e la possibilità di fallimenti nella connessione se un utente si disconnette durante il processo di review.

### Gestione input

I comandi vengono letti tramite `fgets()` con verifica della presenza del newline per rilevare input troppo lunghi; se il buffer viene superato, il contenuto residuo viene scartato leggendo fino al newline successivo; l'inserimento di caratteri vuoti viene gestito correttamente. Il carattere separatore `|` viene esplicitamente vietato nei testi delle card durante la creazione, prevenendo conflitti con il protocollo di comunicazione che utilizza tale carattere come delimitatore di campo. Tutti i comandi sono sempre disponibili, ma vengono validati contestualmente in base allo stato corrente. Per evitare comportamenti indesiderati causati da doppi invii, vengono utilizzati flag (`ack_card_inviata`, `card_done_inviata`) che vengono resettati ad ogni nuova card assegnata e controllati prima dell'invio del comando.

## Esecuzione

### Lavagna (./lavagna)

Comandi disponibili da terminale:

- `SHOW_LAVAGNA` - Visualizza lo stato attuale della lavagna con tutte le card
- `HANDLE_CARD` - Assegna le card in `TO_DO` agli utenti disponibili
- `SEND_USER_LIST` - Invia la lista degli utenti connessi a tutti i client

### Utente (./utente porta)

Comandi utente da terminale:

- `HELLO` - Registra l'utente sulla lavagna (primo comando obbligatorio)
- `SHOW_LAVAGNA` - Richiede e visualizza lo stato della lavagna
- `CREATE_CARD` - Crea una nuova card in `TO_DO` (richiede ID, colonna, testo)
- `ACK_CARD` - Accetta la card assegnata, spostandola in `DOING`
- `REVIEW_CARD` - Invia richieste di review P2P a tutti gli altri utenti
- `REQUEST_USER_LIST` - Richiede la lista degli utenti connessi (viene eseguito automaticamente in `REVIEW_CARD`)
- `CARD_DONE` - Completa la card (solo dopo aver ricevuto tutte le review)
- `PONG_LAVAGNA` - Risponde al `PING` della lavagna
- `QUIT` - Disconnette l'utente dalla lavagna e termina il programma.